

a Runner with a GitLab user entity.

Current GitLab CI job handling mechanism

Before going further, it is helpful to level-set the current job handling mechanism in GitLab CI and GitLab Runners.

- First, a runner associated with a GitLab instance continuously queries the GitLab instance API to check if there is a new job that it could run.
- With every push to a project repository on GitLab with a `.gitlab-ci.yml` file present, the CI service present on the GitLab instance catches the event and triggers a new CI job.
- The CI job enters a pending state in the queue until a Runner requests a job from the instance.
- On the request from a runner to the API for a job, the database is queried to verify that the job parameters matches that of the runner. In other words, when runners poll a GitLab instance for a job to execute they're assigned a job if it matches the specified criteria.
- If the job matches the runner in question, then the GitLab instance connects the job to the runner and changes the job state to running. In other words, GitLab connects the job object with the Runner object.
- A runner can be configured to run un-tagged jobs. Tags are the primary mechanism used today to enable customers to have some control of which Runners run certain types of jobs.
- So while runners are scoped to the instance, group, or project, there are no additional access control mechanisms today that can be expanded on to deny access to a runner based on a user or group identifier.

The current CI jobs queue logic is as follows. Note - in the code we still use the very old build naming construct, but we've migrated from build to job in the product and documentation.

```
ruby
jobs =
  if runner.instancetype?
    jobsforsharedrunner
  elsif runner.grouptype?
    jobsforgrouprunner
  else
    jobsforprojectrunner
  end

jobs = jobs.matchestagids(runner.tags.ids)

unless runner.rununtagged?
  jobs = jobs.withanytags
end
```

Goals

- Implement an initial solution that provides an easy-to-configure and use mechanism to allow, deny or redirect CI job execution on a specific runner entity based on some basic job details

(like user, group or project membership).

Non-Goals

- A re-design of the CI job queueing mechanism is not in the scope of this blueprint.

Proposal

Implement a mechanism, admission controllers, to intercept CI jobs, allowing them to either mutate jobs, validate them or do both. An admission controller is a mutating webhook that can modify the CI job or reject the job according to a policy. The webhook is called before the job is inserted into the CI jobs queue.

Guiding principles

- The webhook payload schema will be part of our public facing APIs.
- We must maintain backwards compatibility when extending the webhook payload.
- Controllers should be idempotent.

How will the admissions controller work?

Scenario 1: I want to deny access to a certain runner.

1. Configure an admissions controller to only accept jobs from specific projects.
1. When a job is created the project information (projectid, jobid, apitoken) will be used to query GitLab for specific details.
1. If the project information matches the allow list, then the job payload is not modified and the job is able to run on the target runner.
1. If the project information does not match the allow list, then the job payload is not modified and the job is dropped.
1. The job tags are not changed.
1. Admission controller may optionally send back an arbitrary text description of why a decline decision was made.

Scenario 2: Large runner fleet with using a common configuration and tags.

Each runner has a tag such as zonea, zoneb. In this scenario the customer does not know where a specific job can run as some users have access to zonea, and some to zoneb. The customer does not want to fail a job that should run on zonea, but instead redirect a job if it is not correctly tagged to run in zonea.

1. Configure an admissions controller to mutate jobs based on userid.
1. When a job is created the project information (projectid, jobid, apitoken) will be used to query GitLab for specific details.
1. If the userid matches then the admissions controller modifies the job tag list. zonea is added to the tag list as the controller has detected that the user triggering the job should have their jobs run IN zonea.

Scenario 3: Runner pool with specific tag scheme, user only has access to a specific subset

Each runner has a tag identifier unique to that runner, e.g. DiscoveryOne, tugNostromo, MVSeamus, etc. Users have arbitrary access to these runners, however we don't want to fail a job on access denial, instead we want to prevent the job from being executed on runners to which the user does not have access. We also don't want to reduce the pool of runners the job can be run on.

1. Configure an admissions controller to mutate jobs based on userid.
1. When a job is created the project information (projectid, jobid, apitoken) will be used to query GitLab for specific details.
1. The admission controller queries available runners with the userid and collects all runners for which the job cannot be run. If this is all runners, the admission controller rejects the job, which is dropped. No tags are modified, and a message is included indicating the reasoning. If there are runners for which the user has permissions, the admission controller filters the associated runners for which there are no permissions.

MVC

Admission controller

1. A single admission controller can be registered at the instance level only.
1. The admission controller must respond within 1 hr.
1. The admission controller will receive individual jobs. The response must contain only responses to that job.
1. The admission controller will receive an API callback for rejection and acceptance, with the acceptance callback accepting mutation parameters.

Job Lifecycle

1. The preparing job state will be expanded to include the validation process prerequisite.

```
mermaid
stateDiagram-v2
    created --> preparing
    state preparing {
        [] --> accept
        [] --> reject
    }
    reject --> failed
    accept --> pending
    pending --> running: picked by runner
    running --> executed
    state executed {
        [] --> failed
        [] --> success
        [] --> canceled
    }
    executed --> created: retry
```

1. When the state is preparing, the mutating webhook payload is sent to the admission

controller asynchronously. This will be retried a number of times as needed.

1. The preparing state will wait for a response from the webhook or until timeout.
1. The UI should be updated with the current status of the job prerequisites and admission
1. For jobs where the webhook times out (1 hour) their status should be set as though the admission was denied with a timeout reasoning. This should be rare in typical circumstances.
1. Jobs with denied admission can be retried. Retried jobs will be resent to the admission controller without tag mutations or runner filtering reset.
1. allowfailure should be updated to support jobs that fail on denied admissions, for example:

```
yaml
job:
  script:
    - echo "I will fail admission"
  allowfailure:
    ondeniedadmission: true
```

1. The UI should be updated to display the reason for any job mutations (if provided) or rejection.
1. Tag modifications applied by the Admission Controller should be persisted by the system with associated reasoning for any modifications, acceptances, or rejections

Payload

1. The payload is comprised of individual job entries consisting of:
 - Job ID.
 - Predefined variables
 - Job tag list.
1. The response payload is comprised of individual job entries consisting of:
 - Job ID.
 - Admission state: accepted or denied.
 - Mutations: additions and removals. additions supplements the existing set of tags, removals removes tags from the current tag list
 - Reason: A controller can provide a reason for admission and mutation.
 - Accepted Runners: runners to be considered for job matching, can be empty to match all runners
 - Rejected Runners: runners that should not be considered for job matching, can be empty to match all runners

Example request

```
json
[
  {
    "id": 123,
    "variables": {
      predefined variables: https://docs.gitlab.com/ee/ci/variables/predefinedvariables.html
      "CIPROJECTID": 123,
      "CIPROJECTNAME": "something",
```

```

    "GITLABUSERID": 98123,
    ...
  },
  "tags": [ "docker", "windows" ]
}
]
[
{
  "id": 245,
  "variables": {
    "CIPROJECTID": 245,
    "CIPROJECTNAME": "foobar",
    "GITLABUSERID": 98123,
    ...
  },
  "tags": [ "linux", "eu-west" ]
}
]
[
{
  "id": 666,
  "variables": {
    "CIPROJECTID": 666,
    "CIPROJECTNAME": "do-bad-things",
    "GITLABUSERID": 98123,
    ...
  },
  "tags": [ "secure-runner" ]
},
]

```

Example response

```

json
[
{
  "id": 123,
  "admission": "accepted",
  "reason": "it's always-allow-day-wednesday"
}
]
[
{
  "id": 245,
  "admission": "accepted",
  "tags": {
    "add": [ "linux", "us-west" ],
    "remove": [...]
  },
}
]

```

```

    "runners": {
      "acceptedids": ["822993167"],
      "rejectedids": ["822993168"]
    },
    "reason": "user is US employee: retagged region; user only has uid on runner 822993167"
  }
]
[
  {
    "id": 666,
    "admission": "rejected",
    "reason": "you have no power here"
  }
]

```

MVC +

1. Multiple admissions controllers on groups and project levels.
1. Passing job definition through a chain of the controllers (starting on the project, through all defined group controllers up to the instance controller).
1. Each level gets the definition modified by the previous controller in the chain and makes decisions based on the current state.
1. Modification reasons, if reported by multiple controllers, are concatenated.
1. Usage of the admission controller is optional, so we can have a chain containing project+instance, project+group+parent group+instance, project+group, group+instance, etc

Implementation Details

GitLab

1. Expand preparing state to engage the validation process via the prerequisite interface.
1. Amend preparing state to indicate to user, via the UI and API, the status of job preparation with regard to the job prerequisites
 1. Should indicate status of each prerequisite resource for the job separately as they are asynchronous
 1. Should indicate overall prerequisite status
1. Introduce a 1 hr timeout to the entire preparing state
1. Add an AdmissionValidation prerequisite to the preparing status dependencies via `Gitlab::Ci::Build::Prerequisite::Factory`
1. Convert the Prerequisite factory and preparing status to operate asynchronously
1. Convert PreparingBuildService to operate asynchronously
1. PreparingBuildService transitions the job from preparing to failed or pending depending on success of validation.
1. AdmissionValidation performs a reasonable amount of retries when sending request
1. Add API endpoint for Webhook/Admission Controller response callback
 1. Accepts Parameters:
 - Acceptance/Rejection
 - Reason String
 - Tag mutations (if accepted, otherwise ignored)

1. Callback encodes one time auth token
1. Introduce new failure reasoning on validation rejection
1. Admission controller impacts on job should be persisted
1. Runner selection filtering per job as a function of the response from the Admission controller (mutating web hook) should be added

Technical issues to resolve

Issue	Resolution
Rule definition for the queue web hook	
What data to send to the admissions controller? Is it a subset or all of the predefined variables?	
Is the queueing web hook able to run at GitLab.com scale? On GitLab.com we would trigger millions of webhooks per second and the concern is that would overload Sidekiq or be used to abuse the system.	

 ## SECTION: engineering/architecture/design-documents/runner_scaling/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab Runner is a core component of GitLab CI/CD. It makes it possible to run CI/CD jobs in a reliable and concurrent environment. It has been initially introduced by Kamil Trzeci-ski in early 2015 to replace a Ruby version of the same service. GitLab Runner written in Go turned out to be easier to use by the wider community, it was more efficient and reliable than the previous, Ruby-based, version.

In February 2016 Kamil Trzeci-ski implemented an auto-scaling feature to leverage cloud infrastructure to run many CI/CD jobs in parallel. This feature has become a foundation supporting CI/CD adoption on GitLab.com over the years, where we now run around 4 million builds per day at peak.

During the initial implementation a decision was made to use Docker Machine:

- > Is easy to use. Is well documented. Is well supported and constantly extended. It supports almost any cloud provider or virtualization infrastructure. We need minimal amount of changes to support Docker Machine: machine enumeration and inspection. We don't need to implement any "cloud specific" features.

This design choice was crucial for the GitLab Runner success. Since that time

the auto-scaling feature has been used by many users and customers and enabled rapid growth of CI/CD adoption on GitLab.com.

We can not, however, continue using Docker Machine. Work on that project was paused in July 2018 and there was no development made since that time (except for some highly important security fixes). In 2018, after Docker Machine entered the "maintenance mode", we decided to create our own fork to be able to keep using this and ship fixes and updates needed for our use case. On September 26th, 2021 the project got archived and the documentation for it has been removed from the official page. This means that the original reason to use Docker Machine is no longer valid too.

To keep supporting our customers and the wider community and to improve our SaaS runners maintenance we need to design a new mechanism for GitLab Runner auto-scaling. It not only needs to support auto-scaling, but it also needs to do that in the way to enable us to build on top of it to improve efficiency, reliability and availability.

We call this new mechanism the "next GitLab Runner Scaling architecture".

Continuing building on Docker Machine

At this moment one of our core products - GitLab Runner - and one of its most important features - ability to auto-scale job execution environments - depends on an external product that is abandoned.

Docker Machine project itself is also hard to maintain. Its design starts to show its age, which makes it hard to bring new features and fixes. A huge codebase that it brings with a lack of internal knowledge about it makes it hard for our maintainers to support and properly handle incoming feature requests and community contributions.

Docker Machine and it integrated 20+ drivers for cloud and virtualization providers creates also another subset of problems, like:

- Each cloud/virtualization environment brings features that come and go and we would need to maintain support for them (add new features, fix bugs).
- We basically need to become experts for each of the virtualization/cloud provider to properly support integration with their API,
- Every single provider that Docker Machine integrates with has its bugs, security releases, vulnerabilities - to maintain the project properly we would need to be on top of all of that and handle updates whenever they are needed.

Another problem is the fact that Docker Machine, from its beginnings, was focused on managing Linux based instances only. Despite that at some moment Docker got official and native integration on Windows, Docker Machine never

followed this step. Nor its designed to make such integration easy.

There is also no support for MacOS. This one is obvious - Docker Machine is a tool to maintain hosts for Docker Engine and there is no native Docker Engine for MacOS. And by native we mean MacOS containers executed within MacOS operating system. Docker for MacOS product is not a native support - it's just a tooling and a virtualized Linux instance installed with it that makes it easier to develop Linux containers on MacOS development instances.

This means that only one of three of our officially supported platforms - Linux, Windows and MacOS - have a fully-featured support for CI/CD auto-scaling. For Windows there is a possibility to use Kubernetes (which in some cases have limitations) and maybe with a lot of effort we could bring support for Windows into Docker Machine. But for MacOS, there is no auto-scaling solution provided natively by GitLab Runner.

This is a huge limitation for our users and a frequently requested feature. It's also a limitation for our SaaS runners offering. We've maintained to create some sort of auto-scaling for our SaaS Windows and SaaS MacOS runners hacking around Custom executor. But experiences from past three years show that it's not the best way of doing this. And yet, after this time, Windows and MacOS runners autoscaling lacks a lot of performance and feature support that we have with our SaaS Linux runners.

To keep supporting our customers and the wider community and to improve our SaaS runners maintenance we need to design a new mechanism for GitLab Runner auto-scaling. It not only needs to support auto-scaling, but it also needs to do that in the way to enable us to build on top of it to improve efficiency, reliability and availability.

Proposal

Currently, GitLab Runner auto-scaling can be configured in a few ways. Some customers are successfully using an auto-scaled environment in Kubernetes. We know that a custom and unofficial GitLab Runner version has been built to make auto-scaling on Kubernetes more reliable. We recognize the importance of having a really good Kubernetes solution for running multiple jobs in parallel, but refinements in this area are out of scope for this architectural initiative.

We want to focus on resolving problems with Docker Machine and replacing this mechanism with a reliable and flexible mechanism. We might be unable to build a drop-in replacement for Docker Machine, as there are presumably many reasons why it has been deprecated. It is very difficult to maintain compatibility with so many cloud providers, and it seems that Docker Machine has been deprecated in favor of Docker Desktop, which is not a viable replacement for us.

This issue contains a discussion about how people are using Docker Machine right now, and it seems that GitLab CI is one of the most frequent reasons for people to keep using Docker Machine.

There is also an opportunity in being able to optionally run multiple jobs in a

single, larger virtual machine. We can't do that today, but we know that this can potentially significantly improve efficiency. We might want to build a new architecture that makes it easier and allows us to test how efficient it is with PoCs. Running multiple jobs on a single machine can also make it possible to reuse what we call a "sticky context" - a space for build artifacts / user data that can be shared between job runs.

- Design a simple abstraction that users will be able to build on top of

Because there is no viable replacement and we might be unable to support all cloud providers that Docker Machine used to support, the key design requirement is to make it really simple and easy for the wider community to write a custom GitLab plugin for whatever cloud provider they might be using. We want to design a simple abstraction that users will be able to build on top, as will we to support existing workflows on GitLab.com.

The designed mechanism should abstract what Docker Machine executor has been doing: providing a way to create an external Docker environment, waiting to execute jobs by provisioning this environment and returning credentials required to perform these operations.

The new plugin system should be available for all major platforms: Linux, Windows, MacOS.

- Migrate existing Docker Machine solution to a plugin

Once we design and implement the new abstraction, we should be able to migrate existing Docker Machine mechanisms to a plugin. This will make it possible for users and customers to immediately start using the new architecture, but still keep their existing workflows and configuration for Docker Machine. This will give everyone time to migrate to the new architecture before we drop support for the legacy auto-scaling entirely.

- Build plugins for AWS, Google Cloud Platform and Azure

Although we might be unable to add support for all the cloud providers that Docker Machine used to support, it seems to be important to provide GitLab-maintained plugins for the major cloud providers like AWS, Google Cloud Platform and Azure.

We should build them, presumably in separate repositories, in a way that they are easy to contribute to, fork, modify for certain needs the wider community team members might have. It should be also easy to install a new plugin without the need of rebuilding GitLab Runner whenever it happens.

- Write a solid documentation about how to build your own plugin

It is important to show users how to build a plugin, so that they can implement support for their own cloud infrastructure.

Building new plugins should be simple and supported with great documentation. We want to design the plugin system in a way that the entry barrier for contributing new plugins is very low.

- Build a PoC to run multiple builds on a single machine

We want to better understand what kind of efficiency can running multiple jobs on a single machine bring. It is difficult to predict that, so ideally we should build a PoC that will help us to better understand what we can expect from this.

To run this experiment we most likely we will need to build an experimental plugin, that not only allows us to schedule running multiple builds on a single machine, but also has a set of comprehensive metrics built into it, to make it easier to understand how it performs.

Details

How the abstraction will look exactly is something that we will need to prototype, PoC and decide in a data-informed way. There are a few proposals that we should describe in detail, develop requirements for, PoC and score. We will choose the solution that seems to support our goals the most.

In order to describe the proposals we first need to better explain what part of the GitLab Runner needs to be abstracted away. To make this easier to grasp these concepts, let's take a look at the current auto-scaling architecture and sequence diagram.

On the diagrams above we see that currently a runner manager runs on a machine that has access to a cloud provider's API. It is using Docker Machine to provision new Virtual Machines with Docker Engine installed and it configures the Docker daemon there to allow external authenticated requests. It stores credentials to such ephemeral Docker environments on disk. Once a machine has been provisioned and made available for the runner manager to run builds, it is using one of the existing executors to run a user-provided script. In auto-scaling, this is typically done using the Docker executor.

Separation of concerns

There are several concerns represented in the current architecture. They are coupled in the current implementation so we will break them out here to consider them each separately.

- Virtual Machine (VM) shape. The underlying provider of a VM requires configuration to know what kind of machine to create. For example, Cores, memory, failure domain, etc... This information is very provider specific.
- VM lifecycle management. Multiple machines will be created and a

system must keep track of which machines belong to this executor. Typically a cloud provider will have a way to manage a set of homogeneous machines. For example, GCE Instance Group. The basic operations are increase, decrease and usually delete a specific machine.

- VM autoscaling. In addition to low-level lifecycle management, job-aware capacity decisions must be made to the set of machines to provide capacity when it is needed but not maintain excess capacity for cost reasons.
- Job to VM mapping (routing). Currently the system assigns only one job to a given a machine. A machine may be reused based on the specific executor configuration.
- In-VM job execution. Within each VM a job must be driven through various pre-defined stages and results and trace information returned to the Runner system. These details are highly dependent on the VM architecture and operating system as well as Executor type.

See also Glossary below.

Current state

The current architecture has several points of coupling between concerns. Coupling reduces opportunities for abstraction (for example, community supported plugins) and increases complexity, making the code harder to understand, test, maintain and extend.

A primary design decision will be which concerns to externalize to the plugin and which should remain with the runner system. The current implementation has several abstractions internally which could be used as cut points for a new abstraction.

For example the Build type uses the `GetExecutorProvider` function to get an executor provider based on a dispatching executor string. Various executor types register with the system by being imported and calling `RegisterExecutorProvider` during initialization. Here the abstractions are the `ExecutorProvider` and `Executor` interfaces.

Within the docker+autoscaling executor the `machineExecutor` type has a `Machine` interface which it uses to acquire a VM during the common Prepare phase. This abstraction primarily creates, accesses and deletes VMs.

There is no current abstraction for the VM autoscaling logic. It is tightly coupled with the VM lifecycle and job routing logic. Creating idle capacity happens as a side-effect of calling `Acquire` on the `machineProvider` while binding a job to a VM.

There is also no current abstraction for in-VM job execution. VM-specific commands are generated by the runner manager using the `GenerateShellScript` function and injected

into the VM as the manager drives the job execution stages.

Design principles

Our goal is to design a GitLab Runner plugin system interface that is flexible and simple for the wider community to consume. As we cannot build plugins for all cloud platforms, we want to ensure a low entry barrier for anyone who needs to develop a plugin. We want to allow everyone to contribute.

To achieve this goal, we will follow a few critical design principles. These principles will guide our development process for the new plugin system abstraction.

General high-level principles

- Design the new auto-scaling architecture aiming for having more choices and flexibility in the future, instead of imposing new constraints.
- Design the new auto-scaling architecture to experiment with running multiple jobs in parallel, on a single machine.
- Design the new provisioning architecture to replace Docker Machine in a way that the wider community can easily build on top of the new abstractions.
- New auto-scaling method should become a core component of GitLab Runner product so that we can simplify maintenance, use the same tooling, test configuration and Go language setup as we do in our other main products.
- It should support multiple job execution environments - not only Docker containers on Linux operating system.

The best design would be to bring auto-scaling as a feature wrapped around our current executors like Docker or Shell.

Principles for the new plugin system

- Make the entry barrier for writing a new plugin low.
- Developing a new plugin should be simple and require only basic knowledge of a programming language and a cloud provider's API.
- Strive for a balance between the plugin system's simplicity and flexibility. These are not mutually exclusive.
- Abstract away as many technical details as possible but do not hide them completely.
- Build an abstraction that serves our community well but allows us to ship it quickly.
- Invest in a flexible solution, avoid one-way-door decisions, foster iteration.
- When in doubts err on the side of making things more simple for the wider community.
- Limit coupling between concerns to make the system more simple and extensible.
- Concerns should live on one side of the plug or the other--not both, which duplicates effort and increases coupling.

The most important technical details

- Favor gRPC communication between a plugin and GitLab Runner.
- Make it possible to version communication interface and support many versions.
- Make Go a primary language for writing plugins but accept other languages too.

- Autoscaling mechanism should be fully owned by GitLab.

Cloud provider autoscalers don't know which VM to delete when scaling down so they make sub-optimal decisions. Rather than teaching all autoscalers about GitLab jobs, we prefer to have one, GitLab-owned autoscaler (not in the plugin).

It will also ensure that we can shape the future of the mechanism and make decisions that fit our needs and requirements.

Plugin boundary proposals

The following are proposals for where to draw the plugin boundary. We will evaluate these proposals and others by the design principles and technical constraints listed above.

Custom provider

In order to reduce the scope of work, we only want to introduce the new abstraction layer in one place.

A few years ago we introduced the Custom Executor feature in GitLab Runner. It allows users to design custom build execution methods. The custom executor driver can be implemented in any way - from a simple shell script to a dedicated binary - that is then used by a Runner through os/exec system calls.

Thanks to the custom executor abstraction there is no more need to implement new executors internally in Runner. Users who have specific needs can implement their own drivers and don't need to wait for us to make their work part of the "official" GitLab Runner. As each driver is a separate project, it also makes it easier to create communities around them, where interested people can collaborate together on improvements and bug fixes.

We want to design the new Custom Provider to replicate the success of the Custom Executor. It will make it easier for users to build their own ways to provide a context and an environment in which a build will be executed by one of the Custom Executors.

There are multiple solutions to implementing a custom provider abstraction. We can use raw Go plugins, HashiCorp's Go Plugin, HTTP interface or gRPC based facade service. There are many solutions, and we want to choose the most optimal one. In order to do that, we will describe the solutions in a separate document, define requirements and score the solution accordingly. This will allow us to choose a solution that will work best for us and the wider community.

This proposal places VM lifecycle and autoscaling concerns as well as job to VM mapping (routing) into the plugin. The build need only ask for a VM and it will get one with all aspects of lifecycle and routing already accounted for by the plugin.

Rationale: Description of the Custom Executor Provider proposal

Taskscaler provider

We can introduce a more simple version of the Machine abstraction in the form of a "Fleeting" interface. Fleeting provides a low-level interface to a homogeneous VM group which allows increasing and decreasing the set size as well as consuming a VM from within the set.

Plugins for cloud providers and other VM sources are implemented via the HashiCorp go-plugin library. This is in practice gRPC over STDIN/STDOUT but other wire protocols can be used also.

In order to make use of the new interface, the autoscaling logic is pulled out of the Docker Executor and placed into a new Taskscaler library.

This places the concerns of VM lifecycle, VM shape and job routing within the plugin. It also places the concern of VM autoscaling into a separate component so it can be used by multiple Runner Executors (not just docker+autoscaling).

Rationale: Description of the InstanceGroup / Fleeting proposal

POC: Merge request

Glossary

- GitLab Runner - the software application that you can choose to install and manage, whose source code is hosted at gitlab.com/gitlab-org/gitlab-runner.
- runners - the runner is the agent that's responsible for running GitLab CI/CD jobs in an environment and reporting the results to a GitLab instance. It /1/ retrieves jobs from GitLab, /2/ configures a local or remote build environment, and /3/ executes jobs within the provisioned environment, passing along log data and status updates to GitLab.
- runner manager - the runner process is often referred to as the Runner Manager as it manages multiple runners, which are the `[[runners]]` workers defined in the `runners config.toml` file.
- executor - a concrete environment which can be prepared and used to run a job. A new executor is created for each job.
- executor provider - an implementation capable of providing executors on demand. Executor providers are registered on import and initialized once when a runner starts up.
- custom executor - works as an interface between GitLab Runner and a set of binaries or shell scripts with environment variable inputs that enable executing CI jobs in any host computing environment. New custom executors can be added to the system without making any changes to the GitLab Runner codebase.
- custom executor provider - a new abstraction, proposed under the custom provider heading in the plugin boundary proposal section above, which allows new executor providers to be created without modifying the GitLab Runner codebase. The protocol could be similar to custom executors or done over gRPC. This abstraction places all the mechanics of producing executors within the plugin, delegating autoscaling and lifecycle management concerns to each implementation.
- taskscaler - a new library, proposed under the taskscaler provider heading in the plugin boundary proposal section above, which is parameterized with a concrete executor provider and a fleeting provider. Taskscaler is responsible for the autoscaling concern and can be used to autoscale any executor provider using any VM shape. Taskscaler is also responsible for the

runner-specific aspect of VM lifecycle and keeps track of how many jobs are using a give VM and how many times a VM has been used.

- fleeting - a new library proposed along with taskscaler which provides abstractions for cloud provider VMs.

- fleeting instance group - the abstraction that fleeting uses to represent a pool of like VMs.

This would represent a GCP IGM or an AWS ASG (without the autoscaling). Instance groups can be increased, decreased or can provide connection details for a specific VM.

- fleeting plugin - a concrete implementation of a fleeting instance group representing a specific IGM or ASG (when initialized). There will be N of these, one for each provider, each in its own project. We will own and maintain the core ones but some will be community supported. A new fleeting plugin can be created without making any changes to the runner, taskscaler or fleeting code bases. This makes it analogous to the custom executor provider in terms of self-service and decoupling, but along a different line of concerns.

- fleeting plugin Google Compute - the fleeting plugin which creates GCP instances. This lives in a separate project from the fleeting and taskscaler.

- fleeting plugin AWS - the fleeting plugin which creates AWS instances. This lives in a separate project from the fleeting and taskscaler.

Status

Status: RFC.

Who

Proposal:

<!-- vale gitlab.Spelling = NO -->

Role	Who
Authors	Grzegorz Bizon, Tomasz Maczukin, Joseph Burnett
Architecture Evolution Coach	Kamil Trzci-ski
Engineering Leader	Elliot Rushton, Cheryl Li
Product Manager	Darren Eastman, Jackie Porter
Domain Expert / Runner	Arran Walker

DRIs:

Role	Who
Leadership	Elliot Rushton
Product	Darren Eastman
Engineering	Tomasz Maczukin

Domain experts:

Area	Who
Domain Expert / Runner	Arran Walker

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-documents/runner_technical_vision/_index.md

{{< engineering/design-document-header >}}

This document is a technical vision, not a product or feature roadmap. It describes how to decompose the GitLab Runner problem space in such a way that it can be reassembled into the products and features we have, the ones we want, and the ones we haven't thought of yet.

> "Design is to take things apart in such a way that they can be put
> back together" - Rich Hickey
> <https://www.infoq.com/presentations/Design-Composition-Performance>
> (3:10)

Objective

The composition and abstractions of this architecture are designed to solve several long-standing difficulties in the GitLab Runner. They will allow faster feature development by allowing external teams to "solve their own problems" with plugins and steps independent of the core runner team. Decomposition along orthogonal dimensions of responsibility instead of "executors" will allow for more simple components with higher test coverage and overall higher reliability. Consistent use of fewer abstractions will allow easier understanding of the codebase and less repeated work.

For a list of known problems see GitLab Runner Technical Problems.

Vision

Environments

GitLab Runner is available for all major operating systems and architectures. It can run thousands of jobs concurrently in local or remote environments. It can autoscale remote environments based on current and future load. It can be adapted to an unbounded number of environments through a self-service plugin mechanism, such that autoscaling works out-of-the-box.

The runner operating system and architecture does not need to match that of the remote environment. GitLab Runner is capable of discovering and reconnecting to jobs running in remote environments, without losing job state. It provides observability of the remote environment capacity and asserts back-pressure in response to resource constraints (such as insufficient capacity).

Job Composition

Jobs are delivered as gRPC payloads. All jobs are composed of steps which are executed by an agent (Step Runner) in the job environment. Steps may be local or remote. Local step versions are determined by the commit SHA. Remote step versions are determined by a secure lockfile stored in the repository. The results of a job are returned over gRPC to GitLab (separately from the logs) and describe the exact parameters of every step executed. Steps can be written to be deterministic, allowing step results to be used to reproduce artifacts, byte-for-byte

Step policy is provided by GitLab and enforced by GitLab Runner and its agent in the job environment. Policy may constrain available steps and their versions, insert required steps, and constrain resources consumed (such as network and disk).

Delegation

Steps which are capable of creating their own environment locally, such as a Docker step, can delegate steps into that environment with a regular run request. Having full control of the results, they can decide what to do with them and then incorporate the results in the overall tree. In this way the final job results capture step execution across environments.

Development

Job payloads container steps and calling parameters can be downloaded from any environment and run locally for debugging. Or jobs can be debugged in-place by inserting breakpoints into steps and connecting via gRPC to see the execution context and interact with the job environment. Steps can be unit tested with a built-in testing framework. They can be published and consumed through public and private catalogs, as well as within the local repository. Steps can be marked as deprecated or defective and consumers are automatically notified.

Federated Ownership

Common aspects of jobs such as checkout, cache and artifacts are implemented as steps, outside GitLab Runner and its agent. Vertical integrations such as artifacts can be owned entirely by a single team. E.g. The team who owns the artifact APIs in GitLab can also own the steps that upload and download from them. GitLab Runner provides an environment and injects the job payload. But it doesn't control exactly what runs in that payload. Ownership of various aspects of job execution is federated to specialized teams. E.g. Artifact signing and secret management are implemented as steps owned by other teams.

Unified Execution

GitLab Runner has one way to execute jobs. All aspects of executors are implemented by means of encapsulation steps (such as Docker) or as pre/post hooks in the environment plugin mechanism. Runner is responsible only for dispatching jobs to environments and connecting to those environments to deliver the job payload and return results.

Executor responsibilities are pulled out entirely to one or more plugins. E.g. "kubernetes" plugin(s) are capable of customizing pods according to job requirements. There are no more built-in services, just service steps which the runner prepends to the job payload.

Even pre-existing "scripts" are wrapped and delivered as step payloads so all CI configuration gains the benefits of a unified steps-based execution model.

Management

GitLab Runner is supported by tools to setup, configure and maintain small, medium and large installations. Fleet management tools implement industry best practices such as blue-green deployments. These tools are used to operate all runners on GitLab.com as well as being available for self-hosted customers. Anyone can contribute to GitLab.com's runner infrastructure.

Default images for remote job environments are publicly available. Tools to build private images are also available and work out-of-the-box. Best practices for GitLab Runner efficiency and reliability are implemented in the shared tool set.

Observability

Time spans for every job, step and sub-step are exported via OpenTelemetry and are available as observability data. Likewise real-world resource consumption (CPU, memory, etc..) is available for each job. Resource consumption is tracked over time and, when an environment permits, resource requests are tailored automatically to the job's need.

Resource metrics are fetched by GitLab Runner through the environment-specific plugin for each job. Time spans are returned to GitLab Runner with step results. Overall capacity and load is aggregated by the autoscaling component within GitLab Runner. This metric data is pushed from GitLab Runner to a gRPC endpoint on GitLab for observability purposes, so it can be proxied to the appropriate time-series database. Load is also returned to the Job Router for autoscaling and routing purposes.

Routing

Job routing and autoscaling decisions are made centrally in a simple, dedicated routing service. Jobs can be routed around outages and capacity limitations. Created jobs are immediately known to the runner autoscaling system, even before the jobs are ready for execution. GitLab can provide policy which affects routing, including preferred regions, providers and instance types.

Jobs can also be routed according to resource requirements and available capacity, reported by individual runners. Jobs can also have a relative priority which will affect queuing behavior.

Architecture

Arrows are the flow of data

Resources

- Vision Walkthrough
- RunUp and gRPC Steps

SECTION: engineering/architecture/design-documents/runner_technical_vision/problems.md

Problems

Codebase

GitLab Runner's codebase hasn't aged too well and has been extended in multiple directions with compromises being made to add new features.

There are very large common and helper packages, for example, which almost everything ties into.

Challenges:

- The poor separation of concerns leads to easy leaky abstractions, furthering the difficulty in rectifying the problem.
- As well as being less than ideal from a development and maintenance perspective, it means we currently have some restrictions. For example, the gitlab-runner-helper binary should be lightweight, but because it imports common, that pulls in an entire world of dependencies.
- GitLab Runner can't target AIX today due to incompatible dependencies, but the helper binary could be supported with an improved separation of concerns.

Abstract Shell

GitLab Runner converts jobs into multiple target scripts that execute within the job environment. These scripts perform predefined tasks, such as fetching Git sources or extracting caches, as well as executing user-provided scripts (beforescript, script, and afterscript from the CI/CD config).

GitLab Runner has built-in support for powershell, pwsh, and bash/POSIX shells, and it employs an abstract shell implementation to convert target scripts accordingly.

Challenges:

- The GitLab Runner manager generates the target script, but jobs often execute in remote environments. This can lead to scripts that are unfit for the target environment. A common problem is with path names.
- Shell behaviour can vary based on the version, OS settings, and script execution method (via a file or stdin). Addressing one edge case often introduces another, resulting in a constant game of whack-a-mole.
- Limited shell support presents challenges, as maintaining and implementing the full feature set for the current supported shells is already a maintenance burden. The removal of the cmd shell, though still widely used, was due to difficulties in maintenance and feature implementation.
- Feature set deviation: Adding new features that rely on the abstract shell is often difficult to achieve consistently across all supported shells at once.

Executors

GitLab Runner's executors handle the logic required to run a job on a target platform.

Challenges:

- Different executors demand unique approaches to address the same issues, such as script execution and logging, often resulting in significant compromises and leaky abstractions.
- Executors focus solely on individual jobs. For instance, with Docker and Kubernetes executors, a new connection is established for each job. This approach is unsuitable for longer-lived routines needed for functions like garbage collection.
- In-built executors cannot be ported to the "custom executor" due to the limitations of custom executor abstractions and the tight coupling between executors and the rest of the code base within the existing architecture.
- For Docker and Kubernetes, a "helper" container facilitates predefined scripts. However, several problems emerge from the helper being outside the environment where user scripts are executed,

typically involving ownership and file permissions.

- Feature set deviation: Some executors have features others lack. For example, web terminal support is something that needs re-implementing for each executor, so not all executors support it.

Job micro-managing

A job is split into multiple stages: getsources, restorecache, downloadartifacts, userscript, afterscript, uploadartifactsonsuccess etc.

GitLab Runner communicates with the job's environment, instructing it to execute each stage's script in an ad hoc manner. It waits for the script to complete, reads the status code (to determine success or failure), and then proceeds to initiate the next stage.

Challenges:

- If the job's environment is remote and the connection drops while executing a script, Runner cannot resume execution. This issue was frequent with the Kubernetes executor, prompting shell scripts to be executed in the background, with log output redirected to a file read by a separate, resumable process. This adds complexity to the executor and is only available for Kubernetes. This intricate change led to numerous customer-discovered edge cases and took several milestones to address.
- Because Runner controls the execution flow, it cannot crash/exit and resume jobs after a restart.

Configuration

Runner's configuration is complex:

- Multiple runner configurations can be defined.
- Several layers:
 - Global configuration
 - Runner configuration
 - Executor configuration
 - Autoscaling configuration

Challenges:

- Numerous configuration options

Runner supports setting variables in the config file, environment variables, and command line inputs. However, the config file is the only method that supports all settings. The twelve-factor methodology for configuration is arguably poor suited to Runner's complex options.

- All executor options are displayed, regardless of the executor being

- configured, making the --help command overwhelming.
- No validation.
- No user-provided validation exists for job-provided configurations. Ad hoc settings, such as the Kubernetes executor's `cpulimitoverwritemaxallowed`, allow a job to specify the CPU limit, but maintaining these fields is a maintenance burden.
- Although configuration files allow for comments, registering a new runner (or anything that triggers a config rewrite) discards the comments.

Job logs

A job log combines data from various sources, including the runner manager, executor, services, user scripts, and more.

Challenges:

- No semantic markup: We only support collapsible sections and ANSI colour codes.

SECTION: engineering/architecture/design-documents/runner_tokens/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab Runner is a core component of GitLab CI/CD that runs CI/CD jobs in a reliable and concurrent environment. Ever since the beginnings of the service as a Ruby program, runners are registered in a GitLab instance with a registration token - a randomly generated string of text. The registration token is unique for its given scope (instance, group, or project). The registration token proves that the party that registers the runner has administrative access to the instance, group, or project to which the runner is registered.

This approach has worked well in the initial years, but some major known issues started to become apparent as the target audience grew:

Problem	Symptoms
-----	-----
-----	-----
-----	-----
-----	-----
-----	-----
-----	-----
-----	-----
Single token per scope	- The registration token is shared by multiple

runners:
- Single tokens lower the value of auditing and make traceability almost impossible;
- Copied in many places for self-registration of runners;
- Reports of users storing tokens in unsecured locations;
- Makes rotation of tokens costly.
- In the case of a security event affecting the whole instance, rotating tokens requires users to update a table of projects/namespaces, which takes a significant amount of time. |
| No provision for automatic expiration | Requires manual intervention to change token.
Addressed in 30942.

|
| No permissions model | Used to register a runner for protected branches, and for any tags. In this case, the registration token has permission to do everything. Effectively, someone taking a possession of registration token could steal secrets or source code.

|
| No traceability | Given that the token is not created by a user, and is accessible to all administrators, there is no possibility to know the source of a leaked token.

|
| No historical records | When reset, the previous value of the registration token is not stored so there is no historical data to enable deeper auditing and inspection.

|
| Token stored in project/namespace model | Inadvertent disclosure of token is possible.

|
| Too many registered runners | It is too straightforward to register a new runner using a well-known registration token.

|
In light of these issues, it is important that we redesign the way in which we connect runners to the GitLab instance so that we can guarantee traceability, security, and performance.

We call this new mechanism the "next GitLab Runner Token architecture".

Proposal

The proposal addresses the issues of a single token per scope and token storage by eliminating the need for a registration token. Runner creation happens in the GitLab Runners settings page for the given scope, in the context of the authenticated user, which provides traceability. The page provides instructions to configure the newly-created runner in supported environments using the existing gitlab-runner register command.

The remaining concerns become non-issues due to the elimination of the registration token.

Comparison of current and new runner registration flow

mermaid

graph TD

subgraph new[New registration flow]

A[GitLab: User creates a runner in GitLab UI and adds the runner configuration]

-->B[GitLab: creates cirunners record and returns
new 'glrt-' prefixed authentication

token| B

B[Runner: User runs 'gitlab-runner register' command with
authentication token to register new runner manager with
the GitLab instance) --> C[Runner: Does a .runnersystemid file exist in
the gitlab-runner configuration directory?]

C -->|Yes| D[Runner: Reads existing system ID] --> F

C -->|No| E[Runner: Generates and persists unique system ID] --> F

F[Runner: Issues 'POST /runner/verify' request
to verify authentication token validity] --> G[GitLab: Is the authentication token valid?]

G -->|Yes| H[GitLab: Creates cirunnermachine database record if missing] -->

J[Runner: Store authentication token in .config.toml]

G -->|No| I[GitLab: Returns '403 Forbidden' error] --> K(gitlab-runner register command fails)

J --> Z(Runner and runner manager are ready for use)

end

subgraph current[Current registration flow]

A[GitLab: User retrieves runner registration token in GitLab UI] --> B'

B'[Runner: User runs 'gitlab-runner register' command
with registration token to register new runner] -->|Runner: Issues 'POST /runner request' to create
new runner and obtain authentication token| C'[GitLab: Is the registration token valid?]

C' -->|Yes| D'[GitLab: Create cirunners database record] --> F'

C' -->|No| E'(GitLab: Return '403 Forbidden' error) --> K'(gitlab-runner register command fails)

F'[Runner: Store authentication token
from response in .config.toml] --> Z'(Runner is ready for use)

end

style new fill:f2ffe6

Using the authentication token in place of the registration token

<!-- vale gitlab.Spelling = NO -->

In this proposal, runners created in the GitLab UI are assigned authentication tokens

prefixed with glrt- (GitLab Runner Token).

<!-- vale gitlab.Spelling = YES -->

The prefix allows the existing register command to use the authentication token in lieu of the current registration token (--registration-token), requiring minimal adjustments in existing workflows.

The authentication token is shown to the user only once - after completing the creation flow - to

discourage unintended reuse.

Given that the runner is pre-created through the GitLab UI, the register command fails if provided with arguments that are exposed in the runner creation form.

Some examples are --tag-list, --run-untagged, --locked, or --access-level as these are sensitive parameters that should be decided at creation time by an administrator/owner.

The runner configuration is generated through the existing register command, which can behave in

two different ways depending on whether it is supplied a registration token or an authentication token in the --registration-token argument:

Token type	Behavior
-----	-----
Registration token	Leverages the POST /api/v4/runners REST endpoint to create a new runner, creating a new entry in config.toml and a systemid value in a sidecar file if missing (.runnersystemid).
Runner authentication token	Leverages the POST /api/v4/runners/verify REST endpoint to ensure the validity of the authentication token. Creates an entry in config.toml file and a systemid value in a sidecar file if missing (.runnersystemid).

Transition period

During a transition period, legacy tokens ("registration tokens") continue to be shown on the GitLab Runners settings page and to be accepted by the gitlab-runner register command. The legacy workflow is nevertheless discouraged in the UI. Users are steered towards the new flow consisting of creating the runner in the UI and using the resulting authentication token with the gitlab-runner register command as they do today. This approach reduces disruption to users responsible for deploying runners.

Reusing the runner authentication token across many machines

In the existing autoscaling model, a new runner is created whenever a new job needs to be executed. This has led to many situations where runners are left behind and become stale.

In the proposed model, a cirunners table entry describes a configuration that the user can reuse across multiple machines, and runner state from each machine (for example, IP address, platform, or architecture) is moved to a separate table (cirunnermachines). A unique system identifier is generated automatically whenever the runner application starts up or the configuration is saved. This allows differentiating the machine in which the runner is being used.

The systemid value complements the short runner token that is used to identify a runner in command line output, CI job logs, and GitLab UI.

Given that the creation of runners involves user interaction, it should be possible to eventually lower the per-plan limit of CI runners that can be registered per scope.

Generating a systemid value

We ensure that a unique system identifier is assigned at all times to a gitlab-runner installation.

The ID is derived from an existing machine identifier such as /etc/machine-id (on Linux) and hashed for privacy, in which case it is prefixed with s.

If an ID is not available, a random string is used instead, in which case it is prefixed with r.

This unique ID identifies the gitlab-runner process and is sent on POST /api/v4/jobs requests for all runners in the config.toml file.

The ID is generated and saved both at gitlab-runner startup and whenever the configuration is saved to disk.

Instead of saving the ID at the root of config.toml though, we save it to a new file that lives next to it - .runnersystemid. The goal for this new file is to make it less likely that IDs get reused due to manual copying of the config.toml file

plain

scpwhDr7zFz4xBJujFeEM

Runner identification in CI jobs

For users to identify the machine where the job was executed, the unique identifier needs to be visible in CI job contexts.

As a first iteration, GitLab Runner will include the unique system identifier in the build logs, wherever it publishes the short token SHA.

Given that the runner can potentially be reused with different unique system identifiers, we should store the unique system ID in the database.

This ensures the unique system ID maps to a GitLab Runner's systemid value with the runner token.

A new cirunners table holds information about each unique runner manager, with information regarding when the runner last connected, and what type of runner it was.

In the long term, the relevant fields are to be moved from the runners into cirunners.

Until the removal milestone though, they should be kept in the runners as a fallback when a matching cirunners record does not exist.

An expected scenario is the case when the table is created but the runner hasn't pinged the GitLab

instance (for example if the runner is offline).

In addition, we should add the following columns to runners:

- a creatorid column to keep track of who created a runner;
- a registrationtype enum column to runners to signal whether a runner has been created using the legacy register method, or the new UI-based method.

Possible values are :registrationtoken and :authenticateduser.

This allows the stale runner cleanup service to determine which runners to clean up, and allows

future uses that may not be apparent.

sql

```
CREATE TABLE cirunners (  
  ...  
  creatorid bigint  
  registrationtype int8  
)
```

A new pcirunnermachinebuilds table joins the cirunnermachines and cibuilds tables, to avoid adding more pressure to those tables.

We might consider a more efficient way to store contactedat tha